

表 1 CW- k NN Mean の $m = 2-7$ 等重み平均. Balanced AUPR-OUT は ID/OOD の重みを 50:50 に補正した値である.

手法	AUROC $\uparrow$	FPR95 $\downarrow$	Bal. AUPR-OUT $\uparrow$
CW- k NN Mean	0.873581	0.570593	0.832092

0.1 提案手法：CW- k NN Mean

要因分解実験から、クラス別標準化よりも、生特徴、クラス別候補集合、および近傍距離の平均集約が性能を支配することが分かった。そこで、これらの要素だけからなる CW- k NN Mean (Class-Wise k -Nearest-Neighbor Mean) を定義する。本手法は特徴の ℓ_2 正規化、次元別標準化、Huber 化、分類 head、および経験分位点較正を用いない。

凍結 backbone から得る生特徴を $z(x) \in \mathbb{R}^D$ とし、ID クラス $c \in \mathcal{C}_{\text{ID}}$ の学習特徴バンクを

$$\mathcal{B}_c = \{z_i \mid y_i = c\} \quad (1)$$

とする。問い合わせ z に対し、 $\mathcal{B}_c$ 内で Euclidean 距離が小さい順に選んだ近傍集合を $N_{k_c}^c(z)$ とする。ここで $k_c = \min(k, |\mathcal{B}_c|)$ である。クラス条件付き距離を

$$d_c(z) = \frac{1}{k_c} \sum_{z_j \in N_{k_c}^c(z)} \|z - z_j\|_2 \quad (2)$$

と定義する。全 ID クラスを候補とし、最も近いクラスまでの距離

$$s_{\text{OOD}}^{\text{CW-}k\text{NN}}(z) = \min_{c \in \mathcal{C}_{\text{ID}}} d_c(z) \quad (3)$$

を OOD score とする。値が大きいほど OOD らしい。AUROC 等の共通評価器へ ID score を渡す場合は

$$s_{\text{ID}}^{\text{CW-}k\text{NN}}(z) = -s_{\text{OOD}}^{\text{CW-}k\text{NN}}(z) \quad (4)$$

を用いる。主設定は生 DINOv2 特徴、 $k = 150$ 、全 ID クラス候補である。

Algorithm 1 CW- k NN Mean による OOD score の計算

Require: 生の学習特徴 $\mathcal{T} = \{(z_i, y_i)\}$, 問い合わせ集合 $\mathcal{Q}$, 近傍数 k

Ensure: 各 $z \in \mathcal{Q}$ の OOD score $s_{\text{OOD}}(z)$

```
1: for all  $c \in \mathcal{C}_{\text{ID}}$  do
2:    $\mathcal{B}_c \leftarrow \{z_i \mid y_i = c\}$ 
3: end for
4: for all  $z \in \mathcal{Q}$  do
5:    $s_{\text{OOD}}(z) \leftarrow \infty$ 
6:   for all  $c \in \mathcal{C}_{\text{ID}}$  do
7:      $k_c \leftarrow \min(k, |\mathcal{B}_c|)$ 
8:      $N_{k_c}^c(z) \leftarrow k_c$  個の最近傍点 in  $\mathcal{B}_c$ 
9:      $d_c(z) \leftarrow \frac{1}{k_c} \sum_{z_j \in N_{k_c}^c(z)} \|z - z_j\|_2$ 
10:     $s_{\text{OOD}}(z) \leftarrow \min(s_{\text{OOD}}(z), d_c(z))$ 
11:   end for
12: end for
13: return  $\{s_{\text{OOD}}(z) \mid z \in \mathcal{Q}\}$ 
```

■実装. 付録へ参照実装を掲載する場合は, preamble に listings を追加し, 以下を使用する. この実装は報告値を生成した knn_raw_classwise_k150_mean 条件と score 配列が完全一致する.

Listing 1 CW- k NN Mean の PyTorch 参照実装.

```
1 import torch
2
3
4 class CWKNNMean:
5     """Class-wise kNN mean distance on raw features."""
6
7     def __init__(self, k=150, batch_size=256, device="cuda"):
8         self.k = k
9         self.batch_size = batch_size
10        self.device = device
11
12    @torch.no_grad()
13    def fit(self, train_features, train_labels):
14        # Keep the frozen backbone features raw: no L2 normalization.
15        x = torch.as_tensor(
16            train_features, dtype=torch.float32, device=self.device
17        )
18        y = torch.as_tensor(train_labels, device=self.device)
19
20        # Build one memory bank for each known ID class.
21        self.classes = torch.unique(y, sorted=True)
22        self.banks = [x[y == c].contiguous() for c in self.classes]
23
24        # Cache squared norms for exact Euclidean-distance computation.
25        self.bank_norms = [(bank * bank).sum(1) for bank in self.banks]
26        return self
27
28    @torch.no_grad()
29    def ood_score(self, features):
30        """Larger values indicate more OOD-like samples."""
```

```

31     query = torch.as_tensor(
32         features, dtype=torch.float32, device=self.device
33     )
34     chunks = []
35
36     # Batch queries to bound the temporary GPU-memory footprint.
37     for start in range(0, len(query), self.batch_size):
38         q = query[start : start + self.batch_size]
39         q_norm = (q * q).sum(1, keepdim=True)
40         best = torch.full(
41             (len(q),), torch.inf, dtype=q.dtype, device=q.device
42         )
43
44         # Compute a mean kNN distance independently for every class.
45         for bank, bank_norm in zip(self.banks, self.bank_norms):
46             distance2 = (
47                 q_norm + bank_norm[None, :] - 2.0 * q @ bank.T
48             ).clamp_min(0.0)
49             k_c = min(self.k, len(bank))
50
51             # Average the k nearest Euclidean distances in this class.
52             class_distance = torch.sqrt(
53                 torch.topk(distance2, k_c, largest=False).values
54             ).mean(1)
55
56             # The nearest class-wise bank defines the final OOD score.
57             best = torch.minimum(best, class_distance)
58         chunks.append(best)
59     return torch.cat(chunks).cpu().numpy()
60
61     def id_score(self, features):
62         """Larger values indicate more ID-like samples."""
63         # The experiment evaluator expects an ID-oriented score.
64         return -self.ood_score(features)

```